Fpt University

TRƯỜNG ĐẠI HỌC FPT

# CAPSTONE PROJECT REPORT

## Report 1 – Project Introduction

– Ho Chi Minh City, October 2026 –

# Table of Contents

I. Record of Changes ..... 3

II. Project Introduction ..... 4

1. Overview ..... 4

1.1 Project Information ..... 4

1.2 Project Team ..... 4

2. Product Background ..... 4

3. Existing Systems ..... 5

3.1 Haravan Omnichannel ..... 5

3.2 Sapo Omnichannel ..... 5

3.3 BigSeller ..... 5

4. Business Opportunity ..... 5

5. Software Product Vision ..... 6

System Actors ..... 6

Feature Tree ..... 7

6. Project Scope & Limitations ..... 8

6.1 Major Features ..... 8

6.2 Limitations & Exclusions ..... 8

7. Detailed Use Case Specifications ..... 9

UC-01: Synchronize Orders from Sales Channels ..... 9

UC-02: Consolidate Orders into Order Groups ..... 10

UC-03: Compute the Packing Plan ..... 11

UC-04: Review and Approve the Packing Plan ..... 13

UC-05: Estimate Packaging and Shipping Cost ..... 14

UC-06: Create Shipment and Print Documents ..... 14

UC-07: Scan and Update Picking / Packing Status ..... 15

UC-08: View Packing Reports and Action History ..... 16

UC-09: Manage Users, Roles and System Configuration ..... 17

# I. Record of Changes

| Date | A* M, D | In charge | Change Description |
|---|---|---|---|
| 04/08/2026 | A | Nguyễn Phương Mỹ Thuận | Initial draft |
| 25/10/2026 | M | Nguyễn Phương Mỹ Thuận | Aligned with the implemented system for Review 2: software type (web; mobile app planned for Iteration 3), sales channels (Lazada + AURELLE web store; Tiki, TikTok Shop, Shopee, Facebook Marketplace out of scope), actors and responsibilities, features FE-01 to FE-12 (added warehouse, shipping, returns and AURELLE features), limitations, and use cases UC-01 to UC-09 rewritten to match the code (packing algorithm, consolidation rules, GHN carrier, reports). |
| 08/11/2026 | M | Nguyễn Phương Mỹ Thuận | Limitations: Tiki adapter status (added 31/10/2026, outside the Review 2 demo); home dashboard now reads live data; number of SRS use cases updated (UC-01 to UC-108). |
| 13/11/2026 | M | Nguyễn Phương Mỹ Thuận | Feature Tree redrawn in the template mind-map style with three levels and checked against the code; FE-06, LI-06 and the mobile notes updated (app in progress, picking screen live); FE-02, FE-07, FE-08 and FE-10 descriptions completed (AI consolidation rules, dashboard, sign-in and notifications, express self-delivery). |

*A - Added M - Modified D - Deleted

# II. Project Introduction

## 1. Overview

### 1.1 Project Information

- Project name: AI-Assisted Omnichannel Order Fulfillment and Packaging Optimization System (OptiPack AI)
- Project code: AOFP
- Group name: FA26SE036
- Software type: Web application (OptiPack staff portal and the AURELLE customer web store). A Flutter mobile application for warehouse staff is under development in Iteration 3: the picking screen (camera scanning, offline queue) and push notifications work with the live back end; the other screens are prototypes.

### 1.2 Project Team

| Full Name | Role | Email | Mobile |
|---|---|---|---|
| Thân Thị Ngọc Vân | Lecturer | vanttn@fpt.edu.vn | 0912656836 |
| Nguyễn Phương Mỹ Thuận | Leader | Thuannpmse171113@fpt.edu.vn | 0377168254 |
| Lê Đức Trung Thi | Backend Developer | thildtde180553@fpt.edu.vn | 0905749864 |
| Huỳnh Quốc Việt | Frontend Developer | viethqse182482@fpt.edu.vn | 0813076315 |
| Phan Huỳnh Hải Phượng | Frontend Developer | haifuong2408@gmail.com | 0708639363 |

## 2. Product Background

Amid the rapid growth of e-commerce, businesses are expanding their distribution channels across a variety of platforms such as Shopee, Lazada, TikTok Shop, Facebook Marketplace and their own websites. However, warehouse operations for this omnichannel selling model are exposing serious limitations caused by fragmented management processes. According to warehouse logistics industry benchmarks, manual order-picking time typically ranges from 5 to 15 minutes per order depending on complexity (source: Logimax WMS, Warehouse Picking Strategies, 2026). Within that range, the team estimates that warehouse staff at a mid-sized e-commerce business (a fictional persona used to illustrate the problem, not an actual contracted client) spend about 5–7 minutes per order simply performing manual tasks such as switching back and forth between systems, cross-checking inventory, and visually scanning to consolidate orders from the same customer.

In addition, physical packaging has become a major bottleneck that drives up costs. At present, packaging staff rely on intuition and personal experience to choose carton sizes and the amount of cushioning material. According to the team's internal estimate (based on qualitative observation at a few mid-sized omnichannel shops; further field research is needed before this can be treated as verified data), roughly 15% to 20% of orders are packed into oversized boxes. This inefficiency not only generates unnecessary packaging waste but may also add an extra 10% to 15% to shipping fees, since carriers charge based on volumetric weight — a pricing method commonly used by carriers in Vietnam (GHTK, GHN and Viettel Post all apply a similar volumetric conversion formula).

Given the time and resource waste described above, the business's operations leadership (a fictional persona used for the scope of this capstone project — the Operations Manager) has raised an urgent requirement to develop a centralized internal management platform. This platform is expected to automate the fulfillment workflow and use optimization algorithms and artificial intelligence (AI) to calculate and recommend physical packaging, thereby optimizing logistics costs and improving daily order-processing productivity.

## 3. Existing Systems

To address the challenge of omnichannel order management, several software platforms are already fairly popular on the market. Below is an analysis of three representative systems in Vietnam, highlighting their strengths and the feature gaps they have not yet addressed.

### 3.1 Haravan Omnichannel

- Description: A leading e-commerce and omnichannel sales management platform in Vietnam (https://www.haravan.com).
- Actors & key features: Mainly serves Store Owners and Warehouse Staff. Notable features include inventory synchronization, automatic order pulling from Shopee, Lazada and TikTok Shop into a single dashboard, and bulk shipping-label printing.
- Strengths: Intuitive interface, fast API data synchronization, and high stability with domestic e-commerce platforms.
- Weaknesses (gap): Haravan mainly solves the "digital information management" problem. It has no module for optimizing physical packaging space (3D bin packing); choosing the box type or packing method is left to staff judgment.

### 3.2 Sapo Omnichannel

- Description: A sales management software and omnichannel ecosystem widely used by retail businesses (https://www.sapo.vn).
- Actors & key features: Administrator, Cashier, Warehouse Staff. Its strength lies in cash-flow management, order-status tracking and deep integration with shipping carriers (GHTK, Viettel Post, GHN).
- Strengths: Strong API connectivity with shipping partners, enabling accurate delivery-status tracking.
- Weaknesses (gap): Although Sapo can estimate shipping fees from a preset price table, it does not analyze each product's length, width and height to recommend a suitable box size. Packaging still depends on intuition, which easily leads to wasted space and cushioning material.

### 3.3 BigSeller

- Description: A SaaS/ERP platform for managing multiple storefronts across e-commerce platforms in Southeast Asia (https://www.bigseller.com).
- Actors & key features: Store Owner, Warehouse Staff. Provides solid support for barcode-based picking and includes a feature for consolidating orders from the same buyer.
- Strengths: Handles high order volumes during Mega Sale events, with a responsive order-consolidation feature.
- Weaknesses (gap): BigSeller consolidates orders well in software, but does not calculate the box and volumetric weight for consolidated orders. The business still bears the risk of paying higher shipping fees if staff choose the wrong box size.

## 4. Business Opportunity

Based on the analysis of existing systems such as Haravan, Sapo and BigSeller, a clear gap emerges: the lack of an automated solution for physical packaging calculation. Current platforms mainly focus on managing digital data (status synchronization, inventory, cash flow) while leaving the packaging step to subjective human judgment.

This limitation opens an opportunity to develop the AOFP system (OptiPack AI) for the business. By applying 3D bin-packing optimization algorithms, with an AI language model that writes step-by-step packing instructions, the system aims to deliver measurable business value:

- Reduced shipping costs: the system recommends the smallest suitable box (or the fewest parcels) that fits all items of an order, so the business avoids paying for excess volumetric weight.
- Lower packaging material costs: boxes, bags and filling materials are chosen by rules and deducted from stock; reusable boxes from returned parcels are used first when allowed.
- Improved warehouse productivity: Packaging Staff follow a 3D packing plan and a step-by-step guide instead of trial-and-error, which is especially valuable during order surges.

Better control over packaging materials also helps the business move toward sustainable packaging and reduce waste.

## 5. Software Product Vision

For Store Owners and warehouse teams who process omnichannel orders and want to optimize packaging, OptiPack AI is an internal web-based management system that automates the fulfillment workflow from order intake to delivery and return. Unlike pure order-synchronization software and intuition-based box selection, the system consolidates orders of the same recipient, guides picking by bin and oldest stock lot, and solves the 3D packing problem with an optimization engine (BRKGA heuristic with an optional CP-SAT proof service) to recommend the boxes, item placement and filling materials for each parcel. An AI language model writes the packing instructions shown to staff; every packing plan is approved by a person before packing starts. A Flutter mobile application for scanning, offline actions and push notifications is under development (Iteration 3).

### System Actors

The system has six actors that interact with it directly: five human staff roles and one system actor (the Packing Optimization Engine). Customers and the AURELLE web administrator use the AURELLE web store, which acts as the second sales channel. Detailed use-case specifications are presented in the SRS (Report 3).

| Actor | Main responsibilities |
|---|---|
| Store Owner | Monitor orders and order groups, set express priority, cancel AURELLE orders as seller, approve or reject return requests, configure packing and shipping settings, manage staff leave, view packing reports and action history. |
| Warehouse Staff | Pick items of assigned order groups by scanning (barcode scanner or manual entry on the web portal), report missing items, receive stock by lot, manage bin stock, receive and inspect returned items. |
| Packaging Staff | Review the 3D packing plan, approve or reject it, pack parcels (scan items, seal, weigh), review parcels whose weight is outside |

| Actor | Main responsibilities |
|---|---|
| | tolerance, decide partial shipment when items are missing. |
| Shipping Coordinator | Compare carrier quotes, create shipments (one GHN order per parcel), correct delivery addresses, print labels and manifests, follow delivery and failed delivery, link, split or merge order groups. |
| Packing Optimization Engine (system actor) | Automatically compute the packing plan of every picked order group (boxes, parcels, item placement, filling materials) and estimate packaging cost; results always require Packaging Staff approval before packing. |
| System Administrator | Manage user accounts and roles, connect and disconnect marketplace shops, configure warehouses, internal SKUs, the packaging catalog (boxes, bags, materials and material rules), carriers and order-sync settings. |

### Feature Tree

The diagram below groups the twelve Major Features (FE-01 to FE-12, detailed in section 6.1) into the system's six functional branches and breaks each feature into its two main sub-features. All items were checked against the source code on 13/11/2026.

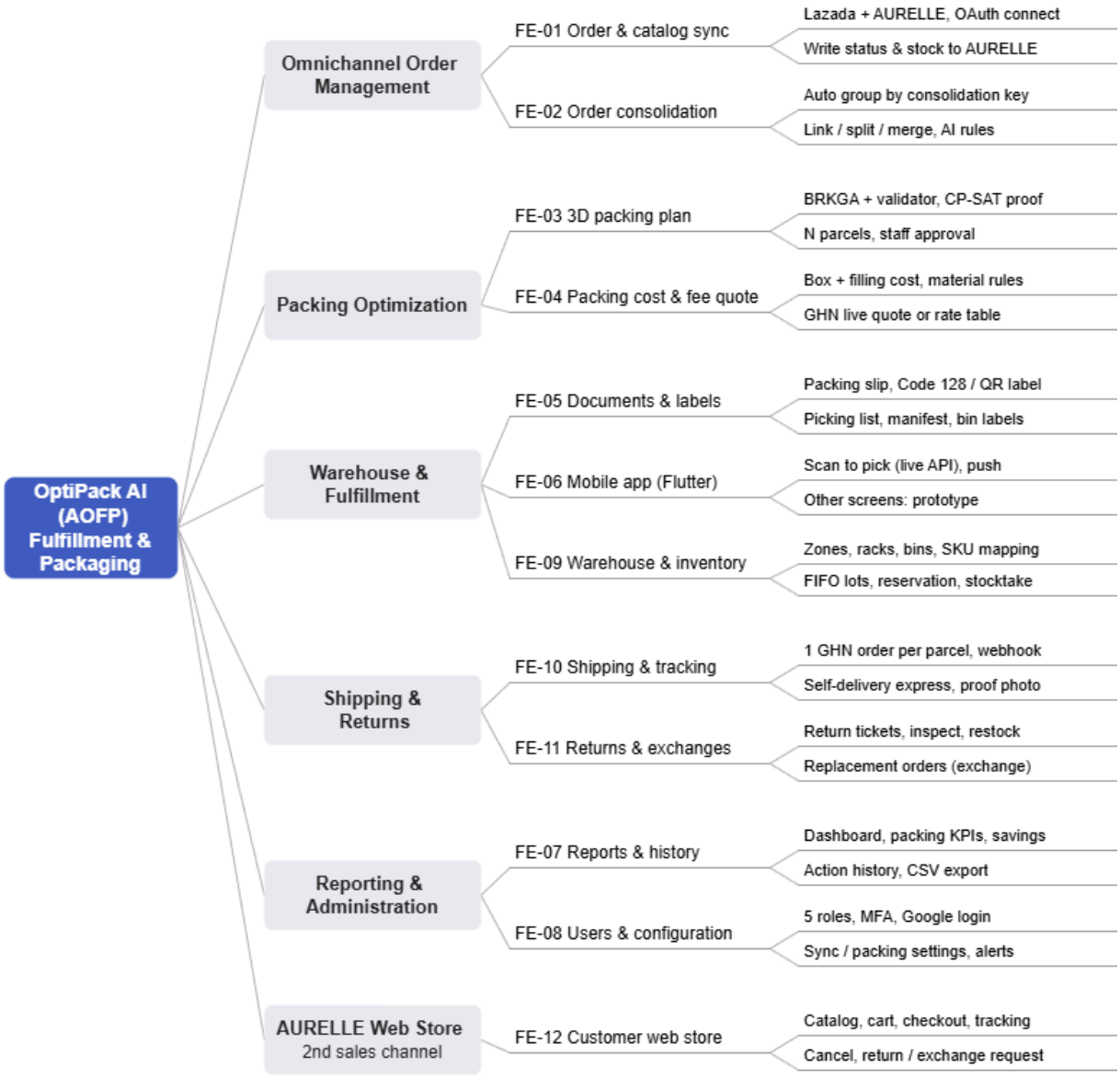



*Figure 1. Feature Tree — hierarchical breakdown of OptiPack AI features*

## 6. Project Scope & Limitations

### 6.1 Major Features

- FE-01: Connect marketplace shops (OAuth) and synchronize orders and product catalog from Lazada and the AURELLE web store; write order status, return status and available stock back to AURELLE.
- FE-02: Automatically consolidate orders of the same recipient (same platform, same shop, same consolidation key) into order groups; allow manual link, split and merge; consolidation rules can be written in plain language and are translated by AI into rule settings that an Admin confirms.
- FE-03: Compute a 3D packing plan for every picked order group (BRKGA heuristic in the back end, optional CP-SAT proof service, one or more parcels per order) with a step-by-step packing guide; Packaging Staff approves, adjusts or rejects the plan before packing.
- FE-04: Estimate packaging cost (box and filling materials) and quote shipping fees per parcel (GHN live quote or a sample rate table, chargeable weight = max(actual weight, volumetric weight)).

- FE-05: Generate PDF documents: packing slip, shipping label (Code 128 and QR), trip manifest, picking list, bin labels, SKU labels and stock receipt.
- FE-06: Mobile application (Flutter) for staff to scan items during picking and packing, with an offline action queue and push notifications (FCM). Status: in progress for Iteration 3 — the picking screen and push notifications run against the live back end, while the packing, shipping and returns screens are prototypes; in the current release scanning on the responsive web portal (barcode scanner or manual entry) remains the main path.
- FE-07: Home dashboard with live figures, packing reports (waiting and packing time, abnormal-weight rate, plans approved without change, cost, incidents, per-staff figures), packaging-material savings, and action history with CSV export.
- FE-08: User management, sign-in (password with lockout, optional MFA, Google sign-in), role-based access control (five staff roles), notifications (in-app, email and mobile push), packing settings, order-sync settings and packaging-catalog management.
- FE-09: Warehouse and inventory management: warehouses, zones, racks and bins with capacity, internal SKUs and seller-SKU mappings, stock receipts by lot (FIFO), stock reservation per order group, stock adjustment, transfer and stocktake.
- FE-10: Shipping and delivery tracking: shipments per order group or same-recipient trip, one GHN order per parcel, carrier status updates (webhook / sync), failed delivery and return to the warehouse; express AURELLE orders can be delivered by the shop's own shipper, with delivery proof photos.
- FE-11: Returns and exchanges: return tickets from AURELLE customer requests and from failed deliveries, approval, receipt, inspection (restock, quarantine, discard) and replacement orders for exchanges.
- FE-12: AURELLE web store: customer registration, product catalog, cart, checkout, order tracking with a 7-step progress bar, order cancellation and return or exchange requests; web administration of products and categories.

### 6.2 Limitations & Exclusions

- LI-01: This is an internal management tool for a single business. The system does not use a multi-tenant architecture and is not a SaaS product.
- LI-02: Order processing is implemented for two sales channels: Lazada (Lazada Open Platform, real shop) and AURELLE (a web store built by the team that exposes an Open API in the Lazada style). Tiki, TikTok Shop, Shopee and Facebook Marketplace are out of scope (Exclusion), based on confirmed technical barriers: Shopee requires a shop to already hold Preferred or Mall Seller status before Open API access is granted; TikTok Shop Partner Center requires a business license and at least one year of operating history; Facebook Marketplace has no public API; Tiki sandbox access was not granted; on 31/10/2026 a Tiki adapter based on a seller Inhouse app (dropship orders) was added and registered, but it is outside the Review 2 demo and is not described in this report. The TikTok Shop adapter file remains in the repository but is not registered.
- LI-03: Lazada has no separate sandbox; tests use the team's real KYC-verified shop. Lazada masks recipient phone, address and sometimes province in order data, so the Shipping Coordinator may need to enter or confirm the delivery address and a demo phone number before booking a carrier.
- LI-04: OptiPack AI does not write order status back to Lazada (read-only by design); status write-back is implemented for AURELLE only.
- LI-05: GHN is the only carrier with a live API integration and is used on the GHN staging (test) environment, so no real pickup or delivery happens; other carriers use sample rate tables.

Delivery status comes from GHN webhooks or from manual status buttons when the webhook cannot reach a local machine.

- LI-06: The mobile application (FE-06) is only partly implemented: the picking screen and push notifications use the live back end; the packing, shipping and returns screens are prototypes with sample data and are not part of the Review 2 demo.

## 7. Detailed Use Case Specifications

This section specifies the 9 main use cases, mapped to the Major Features in section 6.1 and to the actors above. Each use case follows a standard template (Precondition, Trigger, Main Flow, Alternate/Exception Flow, Business Rules, Input/Output Data, Postcondition). The full list of use cases (UC-01 to UC-108) is in the SRS (Report 3).

Note on execution order: the numbering UC-01 to UC-09 follows the feature order, NOT the real operating sequence. The operating sequence implemented in the system is: UC-01 (Sync) → UC-02 (Consolidate, stock reservation and picker assignment) → UC-07 Phase A (Picking) → UC-03 (packing plan computed from the ACTUALLY picked quantity) → UC-04 (Approve) → UC-07 Phase B (Packing session: scan, seal, weigh) → UC-05 (Cost and shipping quote) → UC-06 (Shipment and labels). Reason: the Packaging Staff must see the actually picked items (especially when stock is short) before the packing plan is computed.

### UC-01: Synchronize Orders from Sales Channels

Reference: FE-01

| Field | Description |
|---|---|
| Actor | Main actor: System Scheduler. Secondary actors: System Administrator (connects the shop, can trigger a manual sync), Lazada Open Platform, AURELLE Open API. |
| Description | The system imports new and updated orders from connected Lazada and AURELLE shops and stores them in the internal database. |
| Precondition | The System Administrator has connected at least one shop through OAuth (Lazada or AURELLE) and the shop is active. |
| Trigger | (a) Scheduled job that runs every minute; each shop is synchronized when its configured interval has passed (default 5 minutes, allowed 5–1,440 minutes, Order Sync Settings). (b) For AURELLE: a signed webhook (order changed) triggers a sync of that shop immediately. (c) Manual "Sync" by the System Administrator. |
| Main Flow | 1. The job iterates over all active connected shops whose interval has passed. 2. The system calls GetOrders with update_after = the last successful sync time (first sync: a configured number of days back), page by page (100 orders per page, up to 50 pages). 3. The system fetches the order items in batches of up to 50 orders (GetMultipleOrderItems). 4. Each order is mapped to the internal order model (recipient, items, one of 19 Lazada-compatible statuses; the representative status of an order is the most critical status of its items). 5. The order is upserted by (platform, shop_id, platform_order_id). 6. The order is consolidated into an order group (UC-02). 7. When all pages are read, last_polled_at is moved forward. |

| Field | Description |
|---|---|
| Alternate / Exception Flow | 2a. API error, timeout or rate limit → the error is logged, last_polled_at is not moved, the next run retries the same time window; after repeated failures the Store Owner and Admin receive a "sync failed" notification (at most once every 20 minutes by default). 2b. More than 50 pages → last_polled_at is kept so the remaining orders are read next time. 4a. An order becomes cancelled or lost → its order group is re-checked; a group without fulfillable orders is cancelled automatically (UC-02). 4b. A buyer cancellation request on Lazada (need_cancel_confirm) → the Store Owner and Admin are notified. |
| Business Rules | BR-01: Each order is uniquely identified by (platform, shop_id, platform_order_id) — no duplicates. BR-02: All orders updated after the watermark are imported; only orders in the configured consolidatable statuses join an order group, and orders in non-packable statuses (cancelled, lost, damaged, failed…) are excluded from picking and packing. |
| Input Data | Shop access token (stored encrypted with AES-256-GCM), update_after, offset/limit. |
| Output Data | Order documents (platform, shop_id, platform_order_id, recipient, items[], status, consolidation_key), sync result {fetched, upserted, newlyConsolidated}. |
| Postcondition | New or changed orders appear in the omnichannel order list and are grouped for picking. |
| Integration Notes | Lazada Open Platform REST API with HMAC-SHA256 request signing implemented by the team (no SDK). AURELLE Open API uses the same protocol (shared protocol client) and sends HMAC-signed webhooks; the AURELLE Open API used in the demo runs as a simulator (be/scripts/aurelle-mock-server.ts) that reads AURELLE web-store data. |

### UC-02: Consolidate Orders into Order Groups

Reference: FE-02

| Field | Description |
|---|---|
| Actor | Main actor: System. Secondary actors: Store Owner, Shipping Coordinator (link, split, merge), Warehouse Staff (views the result). |
| Description | Groups orders that go to the same recipient so they are picked, packed and shipped together, reserves stock and assigns a picker. |
| Precondition | An order was imported or updated (UC-01) and its status is consolidatable. |
| Trigger | Immediately during the same sync cycle; a backfill job every 5 minutes creates missing groups and resumes groups stuck before picking. |

| Field | Description |
|---|---|
| Main Flow | 1. The system computes the consolidation key: SHA-256 of the platform and the normalized recipient fields (default: phone + address line + city; the fields are configurable, the phone or the address must be included). 2. The system looks for an open group of the same platform and the same shop with the same key (open = awaiting_packaging or picking by default, configurable; an optional time window can be set). 3. Found → the order joins the group; not found → a new order group is created. 4. For a new group the system reserves stock per SKU in a warehouse, assigns the least-busy Warehouse Staff who is not on leave and moves the group to picking. 5. If the group now contains recipients with different names, the Store Owner, Shipping Coordinator and assigned picker are notified. |
| Alternate / Exception Flow | 4a. Available stock does not cover the group → the group is flagged stock shortage and the Store Owner is notified; it is re-checked automatically after stock-in. 5a. Same recipient on different platforms (or masked Lazada data) → groups are not merged automatically; they share a recipient key and the Shipping Coordinator can link them manually to ship together. 5b. Split or merge is allowed only before the first item of the group is scanned; merge requires the same shop and the same real phone number, and a note is mandatory. |
| Business Rules | BR-03: Orders are consolidated only within the same platform and the same shop. BR-04: An order arriving after its group has passed picking is not added to that group; it forms a new group. |
| Input Data | Order recipient fields, order sync settings (key fields, open statuses, time window). |
| Output Data | order_group (ObjectId id, order_count, consolidation_key, recipient_key, fulfillment_status, assigned_staff_id, warehouse_id, stock_shortage flag). |
| Postcondition | The order group is in picking with an assigned Warehouse Staff and reserved stock (or flagged shortage). |
| Integration Notes | No external API call — processed internally on synchronized data. |

## UC-03: Compute the Packing Plan

Reference: FE-03

| Field | Description |
|---|---|
| Actor | Main actor: Packing Optimization Engine (system actor). |
| Description | Solves the 3D bin-packing problem for every order of a picked order group and proposes boxes, parcels, item placement and filling materials. |
| Precondition | The order group is picked (UC-07 Phase A) and every SKU of the group |

| Field | Description |
|---|---|
| | has a confirmed packaging profile (dimensions, weight, fragility, item category). |
| Trigger | A scheduled job every 10 seconds finds picked groups without an active packing plan. |
| Main Flow | 1. The system splits the actually picked quantities among the orders of the group (oldest order first). 2. For each order the engine runs the BRKGA heuristic with an Empty Maximal Space decoder in the back end (TypeScript), considering only boxes in stock, allowed item orientations, support, stacking load, box weight limit and a 5 mm cushion around fragile items; clothing can be folded in half when this allows a smaller box. 3. The engine minimizes the number of parcels, then the box cost; each parcel contains items of exactly one order. 4. Filling materials are selected by the material rule table (estimate, not placed geometrically). 5. A validator checks every parcel independently before the plan is saved. 6. The plan is saved as ready, the group moves to pending_approval, a Packaging Staff is assigned automatically and notified. 7. For small orders (up to 12 units) the optional CP-SAT service (Python, OR-Tools) runs in the background to prove optimality and may replace the plan with a better validated one. |
| Alternate / Exception Flow | 0a. A SKU has no confirmed packaging profile → the plan fails, the group is flagged packing-blocked and Admin and Store Owner are notified; it is retried after the profile is confirmed. 2a. An item does not fit any box or no box is in stock → the order is marked no_fit with a reason code (item too large, too heavy, out of stock, search budget exhausted). 7a. CP-SAT service unavailable → the BRKGA plan is kept with the label "heuristic". |
| Business Rules | BR-05: Each parcel contains items of one order only. BR-06: Fragile items get a 5 mm cushion in the geometry (configurable in packing settings) and only clothing categories can be folded. |
| Input Data | order_group_id, picked quantities, packaging profiles, box catalog with available stock, material rules, packing settings. |
| Output Data | packing_plan (orders[], parcels[] with box, placements, materials, estimated weight and cost, proof label optimal_global / optimal_in_model / heuristic, lower bound of the parcel count). |
| Postcondition | A packing plan is waiting for approval; it is never applied automatically. |
| Integration Notes | Solver runs in-process in the NestJS back end; CP-SAT is a separate internal HTTP service (packer, FastAPI) and is optional. An LLM (Groq API, model openai/gpt-oss-120b) rewrites the step-by-step packing guide in natural language; it receives only geometric facts (no customer data) and the template text is used when it is unavailable. |

## UC-04: Review and Approve the Packing Plan

Reference: FE-03

| Field | Description |
|---|---|
| Actor | Main actor: Packaging Staff. Secondary actors: System Administrator, Store Owner (notified of rejections). |
| Description | Packaging Staff reviews the 3D plan of each order and approves, adjusts or rejects it before packing — a mandatory human-in-the-loop step. |
| Precondition | A packing plan is ready (UC-03) and the group is pending_approval. |
| Trigger | Packaging Staff opens the plan on the Packing Station screen. |
| Main Flow | 1. Packaging Staff views parcels, boxes, the 3D viewer, materials, cost and the explanation of the result. 2. Packaging Staff approves the plan. 3. The group moves to approved_for_packing. 4. The approval (user, time) is stored in the plan. |
| Alternate / Exception Flow | 2a. Adjust: change the box of a parcel or move an item to another parcel of the same order; every change passes the validator and is logged with a reason. 2b. Recompute with options (exclude boxes, prefer cheaper or fewer parcels). 2c. Reject with a reason code (and a note for "other"); Admin and Store Owner are notified with a 2-working-hour deadline; recovery is recompute, manual packing (approved by another person) or sending the group back to picking. 2d. Approval is refused while any item of any order is not placed. |
| Business Rules | BR-07: Every approve, adjust or reject is tied to the logged-in user. BR-08: A manual plan cannot be approved by the person who created it. |
| Input Data | plan id, expected_version, action, box_code / item / reason (when adjusting or rejecting). |
| Output Data | Plan status approved (or ready after adjustment, rejected), adjustments[] log, approved_by, approved_at. |
| Postcondition | The group is approved_for_packing and the packing session can start. |
| Integration Notes | No external API call; adjustment and rejection logs feed the packing feedback report (UC-08). |

## UC-05: Estimate Packaging and Shipping Cost

Reference: FE-04

| Field | Description |
|---|---|
| Actor | Main actor: System. Secondary actor: Shipping Coordinator. |
| Description | Calculates the packaging cost of each parcel and quotes the shipping fee |

| Field | Description |
|---|---|
| | per parcel for each carrier service. |
| Precondition | The group has an approved or packed packing plan. |
| Trigger | The Shipping Coordinator opens the shipping quote of a packed group. |
| Main Flow | 1. The system reads the parcels (box outer dimensions, estimated or measured weight). 2. Packaging cost = box price + filling materials. 3. For a carrier with a live API (GHN) the system requests a fee preview for every parcel; for a carrier with a rate table it computes chargeable weight = max(actual weight, outer volume ÷ the carrier's volumetric divisor) and looks up the weight tier. 4. The system recommends a service by the configured strategy (cheapest, fastest or fixed). 5. Quotes are shown to the Shipping Coordinator. |
| Alternate / Exception Flow | 3a. GHN does not answer or rejects the address → that service is shown as unavailable with the reason and is not recommended. |
| Business Rules | BR-09: The volumetric divisor is configured per carrier; GHN fees always come from GHN. Rate-table carriers are sample data (flagged as sample rates). |
| Input Data | order_group_id, parcels, carrier catalog, shipping settings, delivery address. |
| Output Data | Per service: fee per parcel, total fee, expected delivery time, availability reason. |
| Postcondition | The coordinator can choose a carrier and service; the chosen cost is stored on each parcel when the shipment is created. |
| Integration Notes | GHN API (fee preview on the staging environment). |

### UC-06: Create Shipment and Print Documents

Reference: FE-05, FE-10

| Field | Description |
|---|---|
| Actor | Main actor: Shipping Coordinator. Secondary actors: System, GHN. |
| Description | Creates the shipment of a packed group (or a trip of several groups of the same recipient), books carrier orders and prints the packing slip, shipping labels and manifest. |
| Precondition | The order group is packed. |
| Trigger | The Shipping Coordinator chooses a carrier and service and clicks "Create shipment". |

| Field | Description |
|---|---|
| Main Flow | 1. The system creates the shipment (internal code SHP-..., trip code for a trip). 2. For GHN, one GHN order is created per parcel with the recipient of that parcel's order (client order code per parcel); the GHN order code and sort code are stored. 3. The shipment moves to awaiting_pickup (GHN) and the order group to shipped. 4. Documents are generated as PDF: packing slip, one shipping label per parcel (carrier tracking code as Code 128 and QR, sender and recipient) and the trip manifest. 5. AURELLE orders receive the status update. |
| Alternate / Exception Flow | 2a. GHN rejects a parcel (e.g. address conflict, invalid phone) → the error is stored on the parcel; the coordinator corrects the address and re-books only the parcels without a tracking code (no duplicate GHN orders). 2b. The GHN order can be cancelled before pickup; the group returns to packed. |
| Business Rules | BR-10: One carrier order per parcel; a trip may only contain groups of the same recipient (same recipient key or manually linked). |
| Input Data | order_group_id(s), carrier_code, service_code, delivery address corrections. |
| Output Data | shipment (status, carrier_parcels[] with carrier order code, fee, expected delivery), PDF files. |
| Postcondition | Labels are ready to be attached to the parcels; delivery tracking starts (UC-07 / FE-10). |
| Integration Notes | GHN create order, cancel and print APIs; PDF generated with pdfkit, bwip-js and qrcode. OptiPack does not call Lazada "Ready to Ship". |

## UC-07: Scan and Update Picking / Packing Status

Reference: FE-06 (web portal in the current release; mobile app in progress for Iteration 3)

| Field | Description |
|---|---|
| Actor | Main actor: Warehouse Staff (picking). Secondary actor: Packaging Staff (packing session, partial decision). |
| Description | Staff scan items with a barcode scanner or manual entry on the web portal to update picking and packing in real time. |
| Precondition | Phase A (Picking): the group is in picking and assigned to the Warehouse Staff. Phase B (Packing): the group is approved_for_packing (UC-04). |
| Trigger | Warehouse Staff opens the picking list of an assigned group; Packaging Staff starts the packing session. |
| Main Flow | Phase A: 1. The picking list shows per SKU the quantity, bin, oldest lot, other bins and the split per order. 2. Staff scan the SKU at the bin. 3. The |

| Field | Description |
|---|---|
| | system checks the SKU belongs to the group, the quantity does not exceed the ordered quantity and the bin has stock, then deducts stock from the oldest lot (FIFO) in one transaction. 4. When every SKU is fully picked, staff confirm and the group moves to picked (triggers UC-03). Phase B: 5. Packaging Staff scans each item into its parcel. 6. Sealing a parcel deducts the box, materials and bags; the measured weight is entered. 7. When the last parcel is sealed the group moves to packed. |
| Alternate / Exception Flow | 2a. SKU not in the group, quantity over the ordered quantity, wrong warehouse or insufficient bin stock → the scan is rejected with an error code. 2b. A repeated scan with the same client event id is not counted twice. 3a. An item cannot be found → "Report missing item": the group moves to partial_needs_review and the Store Owner and Packaging Staff are notified; Packaging Staff decides to ship the available items (short-shipped items are not charged) or restart picking. 6a. Weight outside tolerance (default 20%) → the parcel is held and reviewed by a different person. 6b. "Quick pack" without scanning is refused when scanning is required by settings or the group has several recipient names. |
| Business Rules | BR-11: The group can become picked only when all ordered quantities are scanned, except through the partial decision. |
| Input Data | order_group_id, sku, scanned_quantity, bin_location_id, warehouse_id, scan_method (barcode/manual), client_event_id; parcel_no, weight. |
| Output Data | pick_events, inventory movements, group status picked / partial_needs_review / packed, parcel status sealed / held. |
| Postcondition | The packed group is ready for the Shipping Coordinator. |
| Integration Notes | Web portal (React) on desktop or mobile browser. Mobile app (Flutter) with offline sync is in progress for Iteration 3 (the picking screen already uses the live back end); the back end supports idempotent scans by client_event_id. |

## UC-08: View Packing Reports and Action History

Reference: FE-07

| Field | Description |
|---|---|
| Actor | Main actor: Store Owner. Secondary actors: System Administrator; other staff roles view the history of their own categories. |
| Description | Views packing performance, packaging cost and savings, and the history of business actions. |
| Precondition | The user is logged in with the required role. |
| Trigger | The user opens the Analytics / Dashboard screen or the History screen. |

| Field | Description |
|---|---|
| | |
| Main Flow | 1. The user selects a time range. 2. The system shows the packing report: waiting time and packing time, abnormal-weight rate, plans approved without change, plans that reached the lower bound of parcels, packaging cost, incidents and figures per staff member. 3. The system shows packaging-material savings from reused boxes. 4. The packing feedback report lists frequent adjustments and rejections with suggestions (e.g. re-measure a SKU, review a box). 5. On the History screen the user filters actions by category, date, SKU, order code or actor; Admin and Store Owner export CSV (up to 5,000 rows). |
| Alternate / Exception Flow | 1a. No data for the selected range → an empty state message is shown. 1b. A history source fails → that source is skipped and the others are still returned. |
| Business Rules | BR-12: Reports are computed on request (no cache); the packing report reads at most 5,000 plans per request. Each role sees only its allowed history categories; the login category is visible to Admin only. |
| Input Data | date range (from, to), filters. |
| Output Data | Report figures (JSON), CSV file. |
| Postcondition | The Store Owner has data to support operational decisions. |
| Integration Notes | No external API call. The home dashboard reads all its figures from the back end (GET /dashboard/summary). Limitation: Excel and PDF export of analytical reports is not provided. |

## UC-09: Manage Users, Roles and System Configuration

Reference: FE-08

| Field | Description |
|---|---|
| Actor | Main actor: System Administrator. Secondary actor: Store Owner (packing settings, staff leave, view). |
| Description | Manages user accounts and roles, marketplace connections, the packaging catalog and system settings. |
| Precondition | The user is logged in as System Administrator. |
| Trigger | The Admin opens the Admin screens. |
| Main Flow | 1. The Admin creates staff accounts with one of the five roles; the system e-mails a temporary password that must be changed within 72 hours. 2. The Admin edits, deactivates, reactivates accounts, resets passwords and disables two-factor authentication of a user. 3. The |

| Field | Description |
|---|---|
| | Admin connects or disconnects Lazada and AURELLE shops. 4. The Admin manages boxes, bags, filling materials and the versioned material rules. 5. Admin (and Store Owner for packing settings) edits packing settings (weight tolerance, fragile cushion, preferred plan, parcel limit, scan requirement) and order sync settings; every save creates a new version with author and time. |
| Alternate / Exception Flow | 1a. The e-mail already belongs to an active account → rejected; it belongs to a deactivated account → the Admin is offered "Reactivate" instead of creating a duplicate. 5a. Two people save settings at the same time → the second save is rejected with a version conflict. |
| Business Rules | BR-13: Only the System Administrator manages accounts, marketplace connections and the packaging catalog; Packaging Staff can only approve or adjust a plan per order. |
| Input Data | User fields, role, shop authorization, catalog items, settings. |
| Output Data | Updated configuration, settings versions, activity and login audit logs. |
| Postcondition | The configuration is applied to subsequent processing. |
| Integration Notes | Lazada Open Platform (app console) and AURELLE developer apps for OAuth; marketplace tokens encrypted with AES-256-GCM. |